

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ ІНЖЕНЕРІЇ
ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

Освітній компонент
«Алгоритми та структури даних»

ЗВІТ
з практичної роботи №4

Виконав:
Студент групи КН-25-1
Гнатюк Є.М.
Перевірила:
Доцент кафедри КІЕ
Сидоренко В.М.

Кременчук 2026

Тема: Алгоритми пошуку та їх складність

Мета: опанувати основні алгоритми сортування та навчитись методам аналізу їх асимптотичної складності.

Завдання 1: Оцінити асимптотичну складність алгоритму лінійного пошуку у O -нотації в найгіршому і в найкращому випадку. Як можна покращити алгоритм лінійного пошуку?

1. Оцінка асимптотики лінійного пошуку:

- Найкращий випадок: $O(1)$. Виникає, коли шуканий елемент знаходиться на найпершій позиції у масиву, і пошук завершується за 1 крок.
- Найгірший випадок: $O(n)$. Виникає, коли елемент розташований у самому кінці масиву або взагалі в ньому відсутній, через що алгоритм перевіряє всі n елементів.

2. Модифікації для покращення лінійного пошуку:

- Лінійний пошук із бар'єром: На кінець масиву штучно додається шуканий елемент. Це дозволяє прибрати одну з двох перевірок у циклі (перевірку на вихід за межі масиву), залишаючи лише порівняння самого елемента, що прискорює роботу алгоритму на практиці.
- Метод транспозиції (самоорганізація): Щоразу, коли елемент успішно знайдено, він міняється місцями із попереднім елементом. При частих повторних запитах популярні елементи поступово зміщуються до початку масиву, зменшуючи середній час пошуку.

Завдання 2: Оцінити асимптотичну складність алгоритму бінарного пошуку у O -нотації в найгіршому і в найкращому випадку.

Для успішного застосування бінарного пошуку обов'язковою умовою є те, що вхідний масив має бути попередньо впорядкованим (відсортованим).

1. Найкращий випадок: $O(1)$. Виникає, якщо шукане значення збігається із центральним елементом масиву на найпершій ітерації.

2. Найгірший випадок: $O(\log n)$. Виникає, коли елемент знаходиться на останньому етапі ділення або взагалі відсутній у масиві. На кожному кроці область пошуку зменшується рівно вдвічі, а розділення займає сталий час. За основною теоремою рекурсії це дає логарифмічну складність.

Завдання 3: Побудувати алгоритм тернарного пошуку і оцінити його асимптотичну складність у О-нотації в найгіршому і в найкращому випадку. Який з алгоритмів є оптимальнішим: бінарний чи тернарний? Обґрунтувати відповідь обчисленнями.

1. Опис алгоритму та асимптотика: Тернарний пошук працює аналогічно бінарному, але відсортований масив ділиться не на дві, а на три рівні частини за допомогою двох центральних точок (mid1 та mid2).

- Найкращий випадок: $O(1)$. Елемент збігається з однією з точок поділу на першому кроці.
- Найгірший випадок: $O(\log_3 n)$. Область пошуку на кожному кроці зменшується втричі.

2. Математичне обґрунтування оптимальності (Порівняння):

Для пошуку в найгіршому випадку кількість порівнянь елементів обчислюється як кількість ітерацій, помножена на кількість порівнянь всередині однієї ітерації:

- У бінарному пошуку робиться 1 ітерація розділення (ділення на 2) і до 2 порівнянь всередині циклу. Загальна кількість порівнянь: $2 * \log_2 n$.
- У тернарному пошуку масив ділиться на 3 частини, але всередині циклу потрібно зробити до 4 порівнянь (щоб зрозуміти, в який саме з трьох відрізків потрапляє елемент). Загальна кількість порівнянь: $4 * \log_3 n$.

Переведемо логарифм тернарного пошуку до основи 2 за формулою переходу логарифмів:

$$4 * \log_3 n = 4 * (\log_2 n / \log_2 3) = 4 * (\log_2 n / 1.585) = 2.52 * \log_2 n.$$

Порівняємо підсумкові коефіцієнти: 2 (для бінарного) менше ніж 2.52 (для тернарного).

Висновок: Бінарний пошук є оптимальнішим, оскільки, попри більшу кількість ітерацій, він виконує менше порівнянь усередині кожної ітерації, що зменшує загальний час роботи процесора.

Контрольні відповіді на питання

1. Що таке алгоритм пошуку і чому він важливий у контексті комп'ютерних наук? Це покрокова процедура для знаходження заданого значення або його індексу у структурі даних. Він є фундаментальним, оскільки

пошук даних є однією з найчастіших операцій у будь-яких інформаційних системах та базах даних.

2. Які основні критерії оцінки ефективності алгоритмів пошуку?

- Часова складність: кількість порівнянь та кроків для знаходження елемента.
- Просторова складність: об'єм додаткової пам'яті, яку задіє алгоритм.
- Вимоги до вхідних даних: здатність працювати з невідсортованими списками.

3. Що таке лінійний пошук і як він працює? Це метод послідовного перебору елементів (метод «грубої сили»). Пошук починається з першого елемента і рухається крок за кроком. Якщо поточний елемент збігається із шуканим — повертається його індекс, якщо перебрано весь масив і нічого не знайдено — повертається -1.

4. Які умови повинні бути виконані для успішного застосування бінарного пошуку? Єдина та обов'язкова умова — масив (список) має бути строго заздалегідь впорядкованим або відсортованим за зростанням чи спаданням.

5. Які переваги та недоліки використання бінарного пошуку порівняно з іншими алгоритмами пошуку?

- Переваги: Величезна швидкість роботи на великих масивах, що становить $O(\log n)$, порівняно з лінійним пошуком, де складність $O(n)$.
- Недоліки: Потребує значних часових витрат на попереднє сортування масиву і не підходить для структур, де дані постійно додаються у випадковому порядку.

6. Що таке тернарний пошук і в чому його відмінність від бінарного пошуку? Це алгоритм пошуку у відсортованому масиві, який на кожному кроці ділить масив не на дві, а на три рівні частини, використовуючи два опорні індекси замість одного. Його головна відмінність — зменшення кількості кроків рекурсії за рахунок збільшення кількості важких логічних порівнянь всередині кожного кроку.